List, Tuple

Language spec: https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-220003
Data.List: https://hackage.haskell.org/package/base/docs/Data-List.html
Data.Tuple: https://hackage.haskell.org/package/base/docs/Data-Tuple.html

```
---- List
--
-- 2 constructors: [] and : (list constructor)
[]                -- empty list
[1,2,3,4]

-- list constructor i.e. a function for generating a list
-- list prepend
-- binary infix operator
4:[]               -- [4]
3:(4:[])
2:(3:(4:[]))
1:(2:(3:(4:[])))
1:2:3:4:[]         -- same
1:2:[3,4]
1:[2,3,4]

-- binary constructor
(:) :: a -> [a] -> [a]
(:)             -- [x,...y]
(:) 1           -- [1,...y]
(:) 1 []        -- [1]
(:) 1 [2,3]     -- [1,2,3]
(:) 1 2:3:[]    -- same

-- partial application
(:[])           -- [x]
(:[]) 1         -- [1]
(:1:2:[])       -- [x,1,2]
(:1:[2])        -- same
(:[1,2])        -- same
(:1:[2]) 3      -- [3,1,2]
(1:)            -- [1,...x]
(1:) []         -- [1]
(1:) [2,3]      -- [1,2,3]
(1:2:)          -- [1,2,...x]
(1:2:) []       -- [1,2]
(1:2:) [3,4]    -- [1,2,3,4]
(::[])          -- ERROR. no n-ary constructor i.e. not [x,y]


-- Tuple
(1,2)
(1,2,3,4,5)

-- special case: tuple with one value
-- i.e. Solo
```

```
-- Ref: https://hackage.haskell.org/package/base/docs/Data-Tuple.html
import GHC.Solo
Solo 2              -- a tuple with one value
(2)                 -- NO. Just a number 1
getSolo (Solo 2)    -- 2. gets the value


-- Tuple constructor i.e. a function for creating a tuple
-- n-ary function
(,) :: a -> b -> (a, b)
(,)             -- binary
(,) 1           -- (1,)
(,) 1 2         -- (1,2)
(,,,)           -- 4-ary
(,,,) 1 2 3 4   -- (1,2,3,4)
()              -- NO. A unit, NOT a 0-ary tuple

-- partial application
(1,,)           -- partial
(1,,) 2         -- (1,2,)
(1,,2,,)        -- alternate
(1,,2,,) 3      -- (1,3,2,,)
(1,,2,,) 3 4    -- (1,3,2,4,)
```

Arithmetic sequence [..]

Arithmetic sequence

Generate list using [..] syntax

e.g. [1..10]

e.g. [1,2..10]

Requires **Enum** class.

In fact, just syntactic sugar for Enum's **enumFrom*** methods.

e.g. [1..10] == enumFromTo 1 10

```
[1..10]             -- [1,2,.,10]
enumFromTo 1 10     -- same
[1,3..11]           -- [1,3,5,..,11]
enumFromThenTo 1 3 11
[1,3..10]           -- OK. Cut off where latest match. [1,3,5,..,9]

-- infinite list
-- Lazily evaluated
[1..]               -- [1,2,...]
enumFrom 1
[1,3..]             -- [1,3,5,...]
enumFromThen 1 3
```

List comprehension

Ref: https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-420003.11

List comprehension

Compute a new list from existing lists

i.e. compute = map, filter, etc
3 types of qualifiers: **generator**, **binding**, **guard** (filter)

List comprehension is a special case of the **do expression**.

```
-- list comprehension
[x+y |                    -- [12,14,16,18,20, ... 106,108,110]
  x <- [10,20..100]
  y <- [1..10]
  even x]

-- do
do
  x <- [10,20..100]
  y <- [1..10]
  guard (even x)
  return (x+y)
```

```
---- Basic
l <- [1..10]
[y | y <- l]        -- trivial. [1..10]
[2 * y | y <- l]    -- [2,4,..,20]
```

```
-- as do
do
 y <- l
 return y
do
 y <- l
 return 2*y
```

```
-- 1) generator
-- multiple generators like cross product
[x + y |                     -- multiple generators.
[11,12,13,21,22,23,...92,93,101,102,103]
    x <- [10,20..100],
    y <- [1..3]]             -- iterate over y per x
```

```
-- as do
do
 x <- [10,20..100]
 y <- [1..3]
 return (x+y)
```

```
-- 2) guard aka filter
[2 * y |                    -- filter with a Bool [2,4,6,8]
  y <- [1..10],
  y < 5
```

]

```
-- as do
do
 y <- [1..10]
 guard (y<5)
 return (2*y)
```

```
l = [(1,2),(-1,3),(1,4),(2,5)]
[y | (1,y) <- l]          -- filter by pattern [2,4]
```

```
-- do
-- NOTE: pattern matching allowed only because MonadFail []
do
 (1,y) <- l
 return y
```

```
-- 3) local binding i.e. let
[x |                      -- [2,4,6..20]
  y <- [1..10],
  let x = y * 2]


-- usage: conditional in list
-- i.e. if-else
-- Filter only allows 'if' case
-- Use let
[x |                                -- ["odd","even","odd",...]

  y <- [1..10]
  let x = if even y then "even" else "odd"]
```

```
-- do might be better
do
  y <- [1..10]
  let x = if even y then "even" else "odd"
  return x
```

GHC Extension: OverloadedLists
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/overloaded_lists.html

Data.List.NonEmpty
Ref: https://hackage.haskell.org/package/base/docs/Data-List-NonEmpty.html

e.g.

```
{-# LANGUAGE OverloadedLists #-}

import qualified Data.List.NonEmpty as N
```

```
import Data.List.NonEmpty (NonEmpty(..))

a = 1 :| 2 :: NonEmpty Int
b = [1,2]  :: NonEmpty Int
```

==========

List Utilities

Builtin utilities
Data.List: https://hackage.haskell.org/package/base/docs/Data-List.html

```
-- Data.List
fst :: (a,b) -> a
snd :: (a,b) -> b

curry :: ((a, b) -> c) -> a -> b -> c
uncurry :: (a -> b -> c) -> (a, b) -> c

head :: HasCallstack => [a] -> a
tail :: HasCallstack => [a] -> [a]
last :: HasCallstack => [a] -> a
init :: HasCallstack => [a] -> [a]
(++) :: [a] -> [a] -> [a]

uncons :: [a] -> Maybe (a, [a])
length :: Foldable t => t a -> Int
null :: Foldable t => t a -> Bool

repeat :: a -> [a]
take :: Int -> [a] -> [a]
replicate :: Int -> a -> [a]
cycle :: HasCallstack => [a] -> [a]
iterate :: (a -> a) -> a -> [a]

(!!) :: HasCallstack => [a] -> Int -> a
elem :: (Foldable t, Eq a) => a -> t a -> Bool
elemIndex :: Eq a => a -> [a] -> Maybe Int
elemIndices :: Eq a => a -> [a] -> [Int]
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
findIndex :: (a -> Bool) -> [a] -> Maybe Int
lookup :: Eq a => a -> [(a, b)] -> Maybe b

filter :: (a -> Bool) -> [a] -> [a]
partition :: (a -> Bool) -> [a] -> ([a],[a])

map :: (a -> b) -> [a] -> [b]
reverse :: [a] -> [a]
transpose :: [[a]] -> [[a]]
permutations :: [a] -> [[a]]
```

```
subsequences :: [a] -> [[a]]

intersperse :: a -> [a] -> [a]
intercalate :: [a] -> [[a]] -> [a]

sort :: Ord a => [a] -> [a]
sortOn :: Ord b => (a -> b) -> [a] -> [a]
insert ::
insertBy :: (a -> a -> Ordering) -> a -> [a] -> [a]
sortBy :: (a -> a -> Ordering) -> [a] -> [a]

zip :: [a] -> [b] -> [(a, b)]
zip3 :: [a] -> [b] -> [c] -> [(a, b, c)]
...
zip7
unzip :: [(a, b)] -> ([a], [b])
unzip3 :: [(a, b, c)] -> ([a], [b], [c])
...
unzip7

zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
zipWith3 :: (a -> b -> c -> d) -> [a] -> [b] -> [c] -> [d]
...
zipWith7 ...

-- set operations
nub :: Eq a => [a] -> [a]
delete :: Eq a => a -> [a] -> [a]
(\\) :: Eq a => [a] -> [a] -> [a]
union :: Eq a => [a] -> [a] -> [a]
intersect :: Eq a => [a] -> [a] -> [a]
nubBy :: (a -> a -> Bool) -> [a] -> [a]
deleteBy :: (a -> a -> Bool) -> [a] -> [a]
unionBy :: (a -> a -> Bool) -> [a] -> [a] -> [a]
intersectBy :: (a -> a -> Bool) -> [a] -> [a] -> [a]

maximumBy :: Foldable t => (a -> a -> Ordering) -> t a -> a
minimumBy :: Foldable t => (a -> a -> Ordering) -> t a -> a

-- scan
scanl :: (b -> a -> b) -> b -> [a] -> [b]
scanr :: (a -> b -> b) -> b -> [a] -> [b]
scanr1 :: (a -> a -> a) -> [a] -> [a]
scanl1 :: (a -> a -> a) -> [a] -> [a]
unfoldr :: (b -> Maybe (a, b)) -> b -> [a]

take :: Int -> [a] -> [a]
drop :: Int -> [a] -> [a]
splitAt :: Int -> [a] -> ([a], [a])
takeWhile :: (a -> Bool) -> [a] -> [a]
dropWhile
dropWhileEnd
span :: (a -> Bool) -> [a] -> ([a], [a])
```

```
break :: (a -> Bool) -> [a] -> ([a], [a])
stripPrefix
group :: Eq a => [a] -> [[a]]
inits :: [a] -> [[a]]
tails :: [a] -> [[a]]
groupBy :: (a -> a -> Bool) -> [a] -> [[a]]

isPrefixOf :: Eq a => [a] -> [a] -> Bool
isSuffixOf :: Eq a => [a] -> [a] -> Bool
isInfixOf :: Eq a => [a] -> [a] -> Bool
isSubsequenceOf :: Eq a => [a] -> [a] -> Bool

genericLength
genericTake
genericDrop
genericSplitAt
genericIndex
genericReplicate
...
```

```
---- Tuple
fst (1, 2)        -- 1. First value of a pair.
snd (1, 2)        -- 2
swap (1,2)        -- (2,1)

-- uncurry, curry: convert binary function to unary with tuple
curry :: ((a, b) -> c) -> a -> b -> c
uncurry :: (a -> b -> c) -> (a, b) -> c
uncurry (*)          -- binary -> unary with tuple
uncurry (*) (2,3)    -- 2*3=6
(*) 2 3              -- same
curry (uncurry (*))  -- inverse. i.e.


---- List
--
head [1..10]           -- 1
tail [1..10]           -- [2..10] i.e. all except head
last [1..10]           -- 10
init [1..10]           -- [1..9] i.e. all except last
(++) :: [a] -> [a] -> [a]
x ++ y                 -- concatenate
singleton 1            -- [1]

-- uncons: inverse of list constructor (:)
uncons :: [a] -> Maybe (a, [a])
list=[1..10]
x:xs = list                    -- x=1 xs=[2..9]
Just (x:xs) = uncons list      -- same
uncons list                    -- Just (1,[2..9])
uncons []                      -- Nothing
```

```
-- length, null: see Foldable notes
length :: Foldable t => t a -> Int
null :: Foldable t => t a -> Bool
length [1]        -- 1
length []         -- 0
null []           -- True. Test empty


-- infinite lists
--
repeat 1             -- [1,1,1...]. Infinite of same value.
take 2 (repeat 1)    -- [1,1]
replicate 2          -- 2 copies of the same value i.e. like
replicate, but finite
replicate 2 1        -- [1,1]
cycle                -- Infinite of same pattern
cycle [1,2]          -- [1,2,1,2,1,2...]
cycle []             -- ERROR
iterate              -- Apply function recursively.
iterate (*2) 1       -- [1,2,4,8,16...]


---- Index
--
(!!) :: [a] -> Int -> a
list!!0        -- list[0]
list!!1        -- list[1]
list!!(-1)     -- ERROR
list!!10000    -- ERROR if out of bounds
[1,1..]!!0     -- 1 i.e works for infinite


---- Search
--
-- elem: search by value
-- elem, notElem: see Foldable notes
elem :: (Foldable t, Eq a) => a -> t a -> Bool
elem 1                -- Look for 1 in list
elem 1 [1..10]        -- True
elem 1 $ [1,1..]      -- True. i.e. works for infinite
elem 2 $ [1,1..]      -- BAD. Never stops.
notElem 1 [1..10]     -- False

-- non-Foldable
elemIndex :: Eq a => a -> [a] -> Maybe Int
elemIndex 3            -- find first index. Nothing if not found.
elemIndex 3 [2,1,3]    -- Just 2
elemIndex 3 [3,3,3]    -- Just 0
elemIndex 3 [1,2]      -- Nothing
elemIndices :: Eq a => a -> [a] -> [Int]
elemIndices 3          -- return all indices
elemIndices 3 [2,3,3]  -- [1,2]
elemIndices 3 []       -- []
```

```
-- find: search by predicate
-- i.e. f(x)=Bool
--
-- find: see Foldable notes
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
find (==1) [1..10]          -- Just 1
-- non-Foldable
findIndex :: (a -> Bool) -> [a] -> Maybe Int
findIndex (==1) [2,2,1]     -- Just 1
findIndices :: (a -> Bool) -> [a] -> [Int]
findIndices (==1) [2,2,1]   -- [1]


---- filter
--
-- See Foldable notes
filter (<5)
filter (<5) [1..10]
partition (<5)            -- similar to filter, but also provide
non-matches
partition (<5) [1..10]    -- ([1..4], [5..10])
--
lookup :: Eq a => a -> [(a, b)] -> Maybe b
lookup (1)                -- like find, but for hashtables i.e.
pairs
lookup (1) [(1,2),(2,3)]  -- Just 2 i.e. match (1,2)
lookup (3) [(1,2),(2,3)]  -- Nothing


---- Map + transform
--
map f x
reverse x
transpose [[1,2,3],[4,5,6]]    -- [[1,4],[2,5],[3,6]]
```

```
1 2 3    -->  1 4
4 5 6         2 5
              3 6
1 2 3
4 5 6 7  -->  1 4
              2 5
              3 6
              7
```

```
transpose [[1,2,3],[4,5,6,7]]  -- [[1,4],[2,5],[3,6],[7]]
-- Combinatorial
permutations [1..3]       -- all rearrangements  [[1,2,3],[2,1,3],
[3,2,1],[2,3,1],[3,1,2],[1,3,2]]
subsequences [1..3]       -- all subsequences [[1],[2],[3],[1,2],
[2,3],[1,2,3]]

-- Misc
intersperse :: a -> [a] -> [a]        -- add between each item
intersperse 10 [1..5]                 -- [1,10,2,10,...]
```

```
intersperse ',' "abc"                  -- "a,b,c"
intercalate :: [a] -> [[a]] -> [a]
intercalate "->" ["ab","cd",ef"]      -- String join "ab->cd->ef"


---- Sort
sort :: Ord a => [a] -> [a]
sort [2,3,1]                           -- [1,2,3] Sort by Ord
-- sort with custom value
sortOn :: Ord b => (a -> b) -> [a] -> [a]
sortOn fst                             -- Pick value to use for
sorting. e.g. fst=sort by first value of pair
sortOn fst [(2,10),(3,30),(1,20)]     -- [(1,20),(2,10),(3,30)]
insert

-- sort by custom compare
-- i.e. generalization of sort/sortOn
insertBy :: (a -> a -> Ordering) -> a -> [a] -> [a]
sortBy :: (a -> a -> Ordering) -> [a] -> [a]
sortBy f                    -- f(x,y) = EQ|GT|LT

sortBy compare              -- same as `sort`
sortBy (comparing fst)      -- same as `sortOn fst`


----  zip*: combine two (or more) lists into one
--
-- zip: combine with tuples
zip :: [a] -> [b] -> [(a, b)]
zip [1..3] [10..13]         -- [(1,10),(2,11),...]
zip [1] [10..13]            -- [(1,10)]. Only shorter.
zip [1..3] [10]             -- [(1,10)]. Only shorter.
zip3 :: [a] -> [b] -> [c] -> [(a, b, c)]        -- zip 3 lists
...
zip7                                            -- up to 7 lists

-- unzip: inverse of zip
-- i.e. split tuple
unzip :: [(a, b)] -> ([a], [b])
unzip                        -- reverse of zip. i.e. [(a,b)] ->
([a],[b])
unzip [(1,2),(3,4)]          -- ([1,3],[2,4])
unzip (zip x y)              -- (x,y)
unzip [1]                    -- ERROR. Not a tuple
unzip [(1,2,3])              -- ERROR.
unzip3 :: [(a, b, c)] -> ([a], [b], [c])      -- 3-tuple
...
unzip7

-- zipWith: custom combine
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
zipWith                      -- zip with a custom function f(x,y)
zipWith (+) [1..3] [10..13]  -- [1+10=11,2+11=13,...]
zipWith (,)                  -- same as zip
```

```
zipWith3 :: (a -> b -> c -> d) -> [a] -> [b] -> [c] -> [d]
  -- zip 3 lists
...
zipWith7 ...
 -- up to 7 lists


---- Set operations
--
-- elements
nub :: Eq a => [a] -> [a]
nub                -- i.e. unique
nub [1,1,3,4]      -- [1,3,4]
delete :: Eq a => a -> [a] -> [a]
delete 1 [1,2,3]   -- delete first found. [2,3]
delete 1 [1,1,1]   -- [1,1]
delete 1 [1,1]     -- [1]

-- subsets
(\\) :: Eq a => [a] -> [a] -> [a]
[1,2,3] \\ [1,3,5] -- [1] set minus
[1,1,1] \\ [1]     -- [1,1] Delete duplicates only if the
subtractor also has duplicates
[1,1,1] \\ [1,1]   -- [1]
[1,1,1] \\ [1,1,1] -- []
union :: Eq a => [a] -> [a] -> [a]
union [1,2,3] [3,4,5]       -- [1,2,3,4,5] union
union [1,1,1] [1]           -- [1,1,1] duplicates in first are kept
union [1] [1,1,1,1]         -- [1] duplicates in second are dropped
intersect [1,2,3] [2,3,4]   -- [2,3]
intersect :: Eq a => [a] -> [a] -> [a]
intersect [1,1,1] [1]       -- [1,1,1] duplicates in first are kept
intersect [1] [1,1,1]       -- [1] duplicates in second are dropped

-- *By: use a boolean comparer
-- i.e.
--   (a -> a -> Bool)
ceilingEq x y = (==) `on` ceiling     -- i.e. == ceiling
nubBy :: (a -> a -> Bool) -> [a] -> [a]
nubBy ceilingEq [1.1,1.2,1.3,2.1]     -- [1.1,2.1]
nubBy (==)                            -- same as nub
deleteBy ...
unionBy
intersectBy


---- Folds
--
-- See Foldable notes
foldr ...
foldr1 ...
foldl ...
foldl' ...         -- strict foldl
foldl1 ...
```

```
foldl1' ...        -- strict foldl1

concat [[1,2],[3,4]]              -- [1,2,3,4]
concatMap (\x->[1..x]) [1..3]     -- [1,1,2,1,2,3] i.e. map +
concat
and [True, True]        -- True
or [False, True]        -- True
any even [1..10]        -- True
all even [1..10]        -- False
sum [1..10]             -- 55
product [1..10]
maximum [1..10]         -- 10
minimum [1..10]         -- 1

-- *By: use custom compare
-- i.e. not Ord.compare
maximumBy :: Foldable t => (a -> a -> Ordering) -> t a -> a
minimumBy :: Foldable t => (a -> a -> Ordering) -> t a -> a
maximumBy f              -- f(x,y) = EQ|GT|LT
maximumBy compare        -- use Ord.compare   i.e. same as maximum


---- scan: like fold, but also return each step
-- NOT foldable
--
-- scanl: from left to right
--   scanl f z [x1,x2...] = [z, f z x, f (f z x) x1, ...]
scanl :: (b -> a -> b) -> b -> [a] -> [b]
scanl (+) 0 [1..3]         -- [0,1,3,6]
foldl (+) 0 [1..3]         -- 6
last (scan l ... )           -- same as foldl
scanl (\accum x -> x:accum) [] [1,2,3]     -- [[],[1],[2,1],
[3,2,1]]

-- scanr
scanr :: (a -> b -> b) -> b -> [a] -> [b]
scanr (+) 0 [1..3]        -- [6,5,3,0]
foldr (+) 0 [1..3]        -- 6
head (scanr ...)            -- same as foldr
foldl (\accu x -> [f x (last accu)] ++ accu) [x0]
scanr (\x accum -> x:accum) [] [1,2,3]     -- [[1,2,3],[2,3],[3],
[]]

--
scanr1 :: (a -> a -> a) -> [a] -> [a]
scanl1 :: (a -> a -> a) -> [a] -> [a]


-- Accumulation
-- See Foldable notes
mapAccumR ...
mapAccumL ...
```

```
-- Unfold
--
unfoldr :: (b -> Maybe (a, b)) -> b -> [a]


-- Sublist
--
take :: Int -> [a] -> [a]
drop :: Int -> [a] -> [a]
splitAt :: Int -> [a] -> ([a], [a])
takeWhile :: (a -> Bool) -> [a] -> [a]
dropWhile
dropWhileEnd
span :: (a -> Bool) -> [a] -> ([a], [a])
break :: (a -> Bool) -> [a] -> ([a], [a])
stripPrefix
group :: Eq a => [a] -> [[a]]
inits :: [a] -> [[a]]
tails :: [a] -> [[a]]

-- *By: use custom compare
-- i.e. not Ord.compare
groupBy :: (a -> a -> Bool) -> [a] -> [[a]]
groupBy f               -- f(x,y) = EQ|GT|LT
groupBy compare         -- use Ord.compare   i.e. same as group


-- Predicates
--
isPrefixOf
isSuffixOf
isInfixOf
isSubsequenceOf


-- Generic
genericLength
genericTake
genericDrop
genericSplitAt
genericIndex
genericReplicate


   ============================================================
```